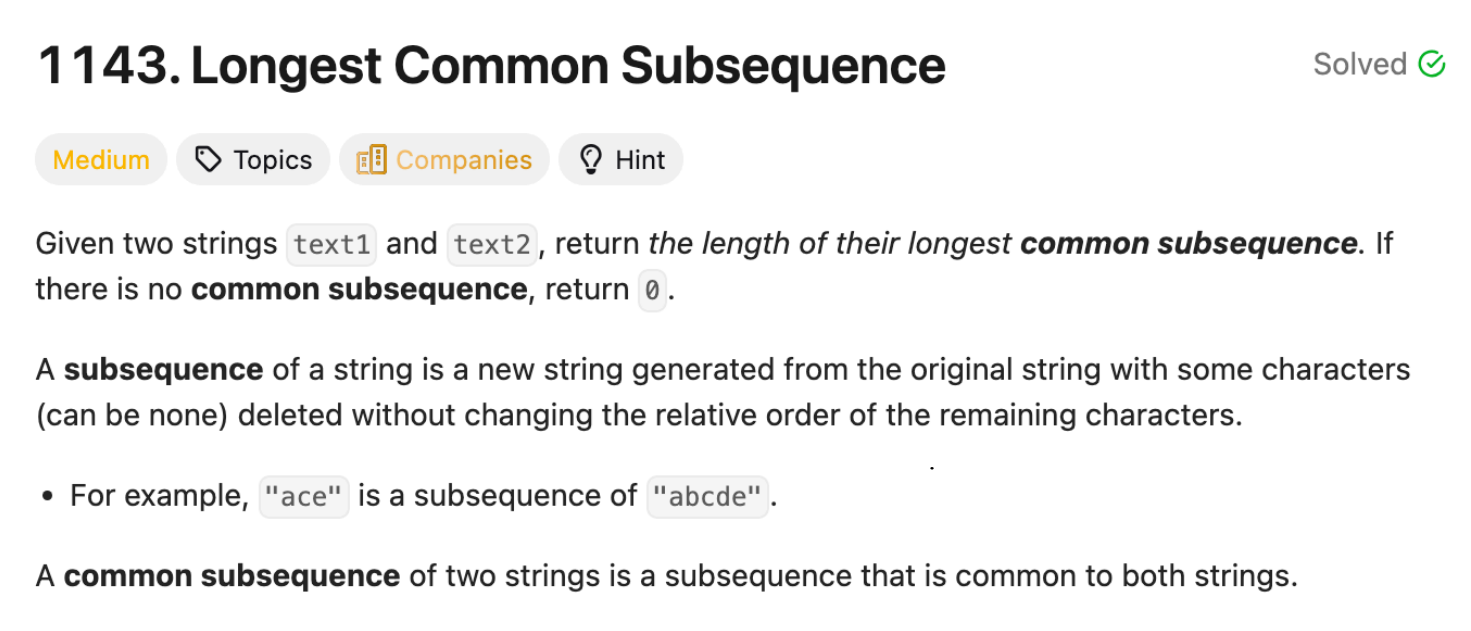
Lec-30

04/08/26 10:21 AM



1143. Longest Common Subsequence

Solved

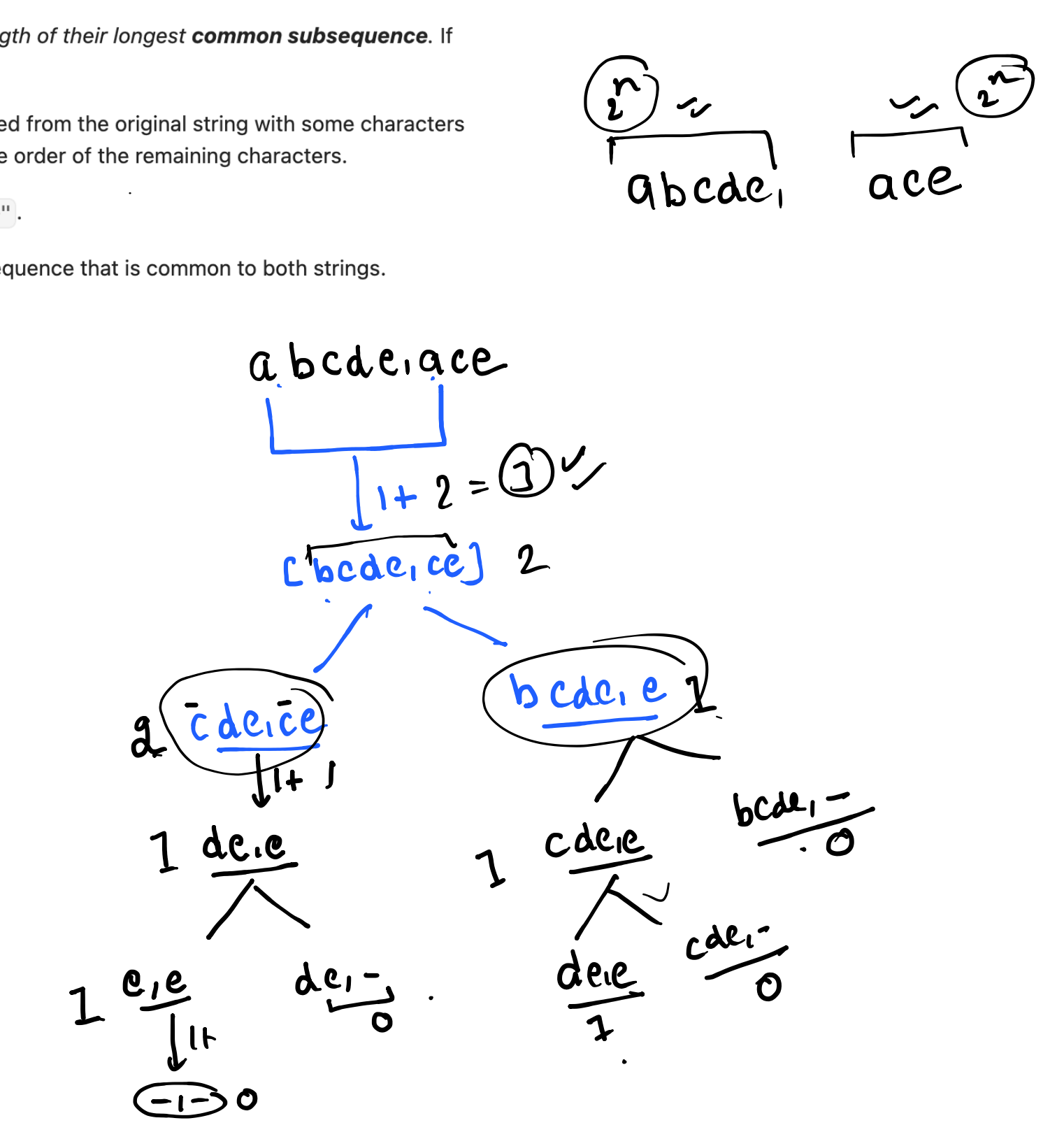
Medium Topics Companies Hint

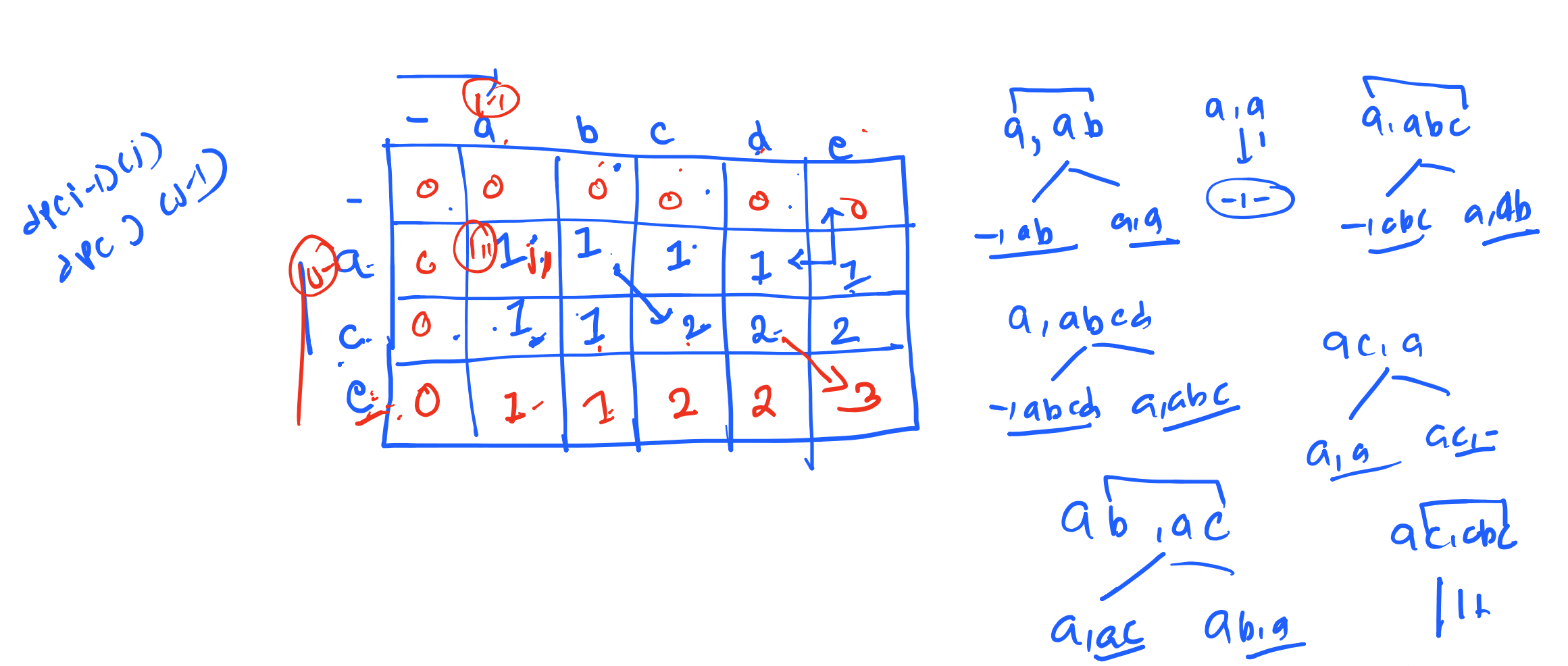
Given two strings `text1` and `text2`, return *the length of their longest* ***common subsequence***. If there is no **common subsequence**, return `0`.

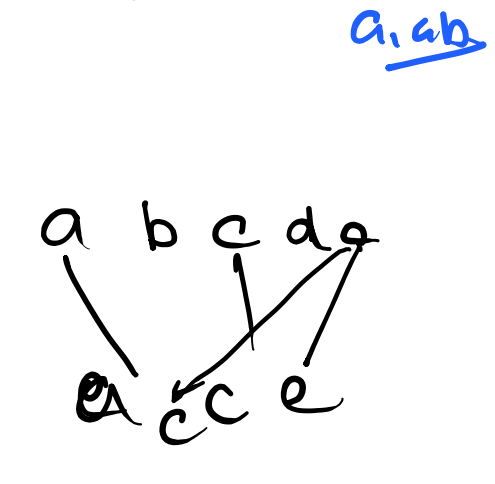
A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

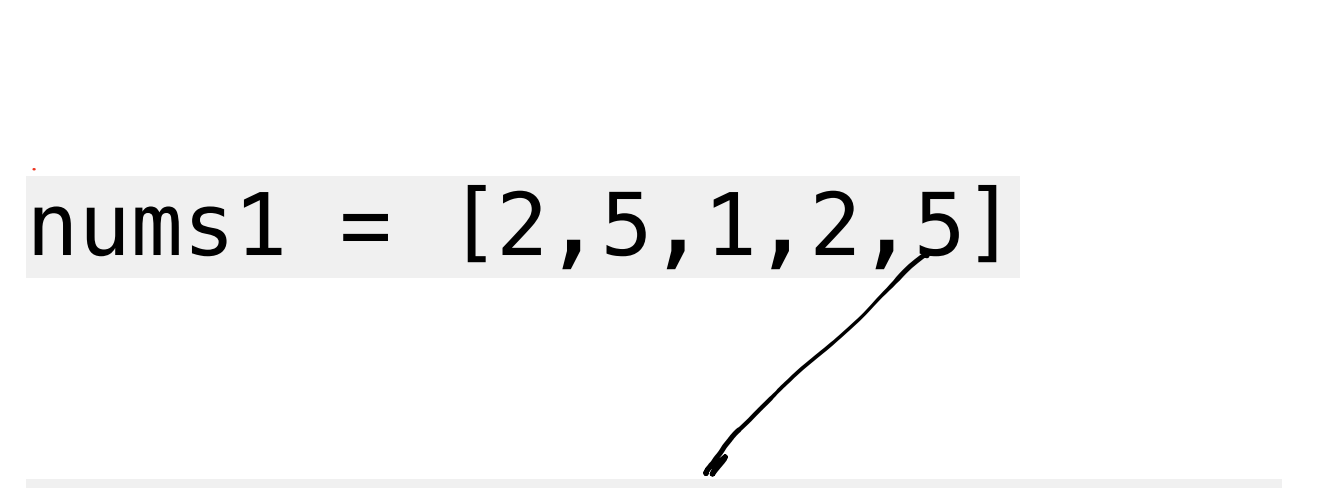
- For example, `"ace"` is a subsequence of `"abcde"`.

A **common subsequence** of two strings is a subsequence that is common to both strings.


2^n
abcde,
ace
2^n
abcde,ace
1+2 = 3
[bcde,ce] 2
2 cde,ce
bcde,e 1
1 de,e
1 cde,
bcde,−
0
1 e,e
de,−
0
de,e
1
cde,−
0
−1− 0

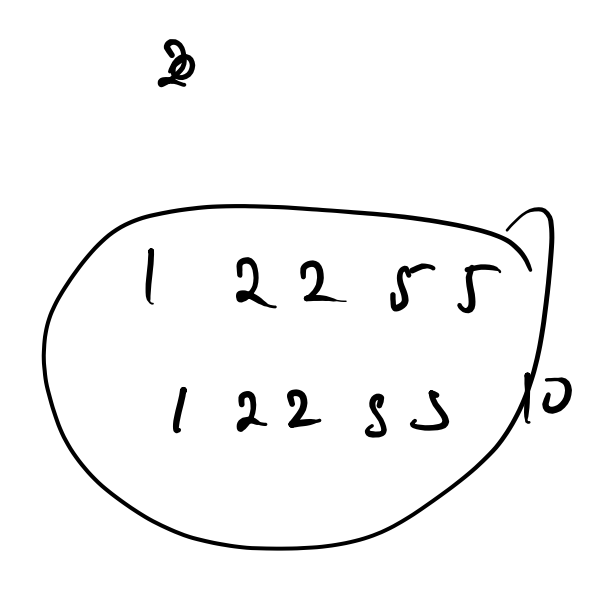

dp[i−1][j]
dp[i][j−1]
a b c d e
a, ab
a, a
a, abc
a, abcd
ac, a
ab, ac
ac, abc
a, ab

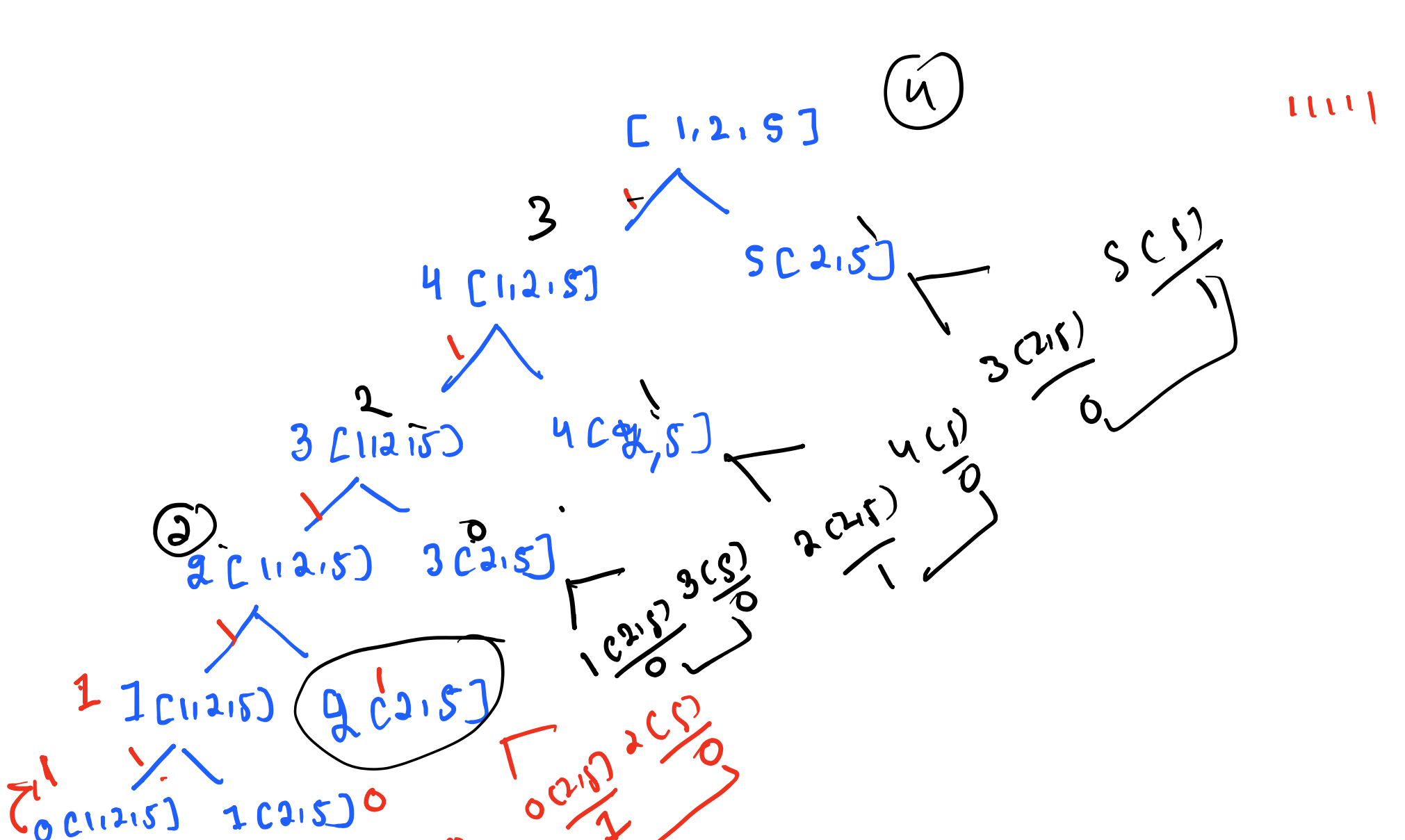


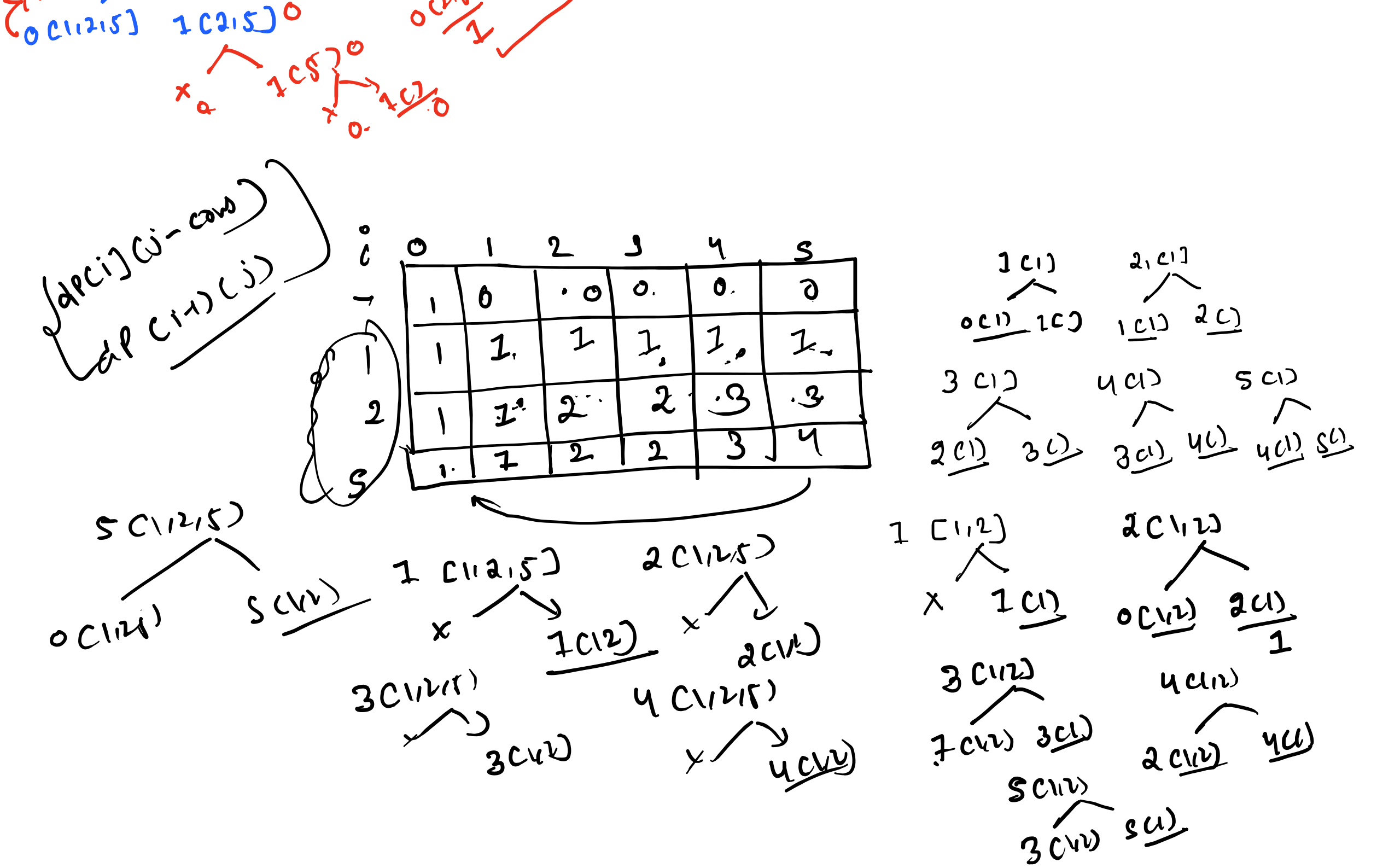


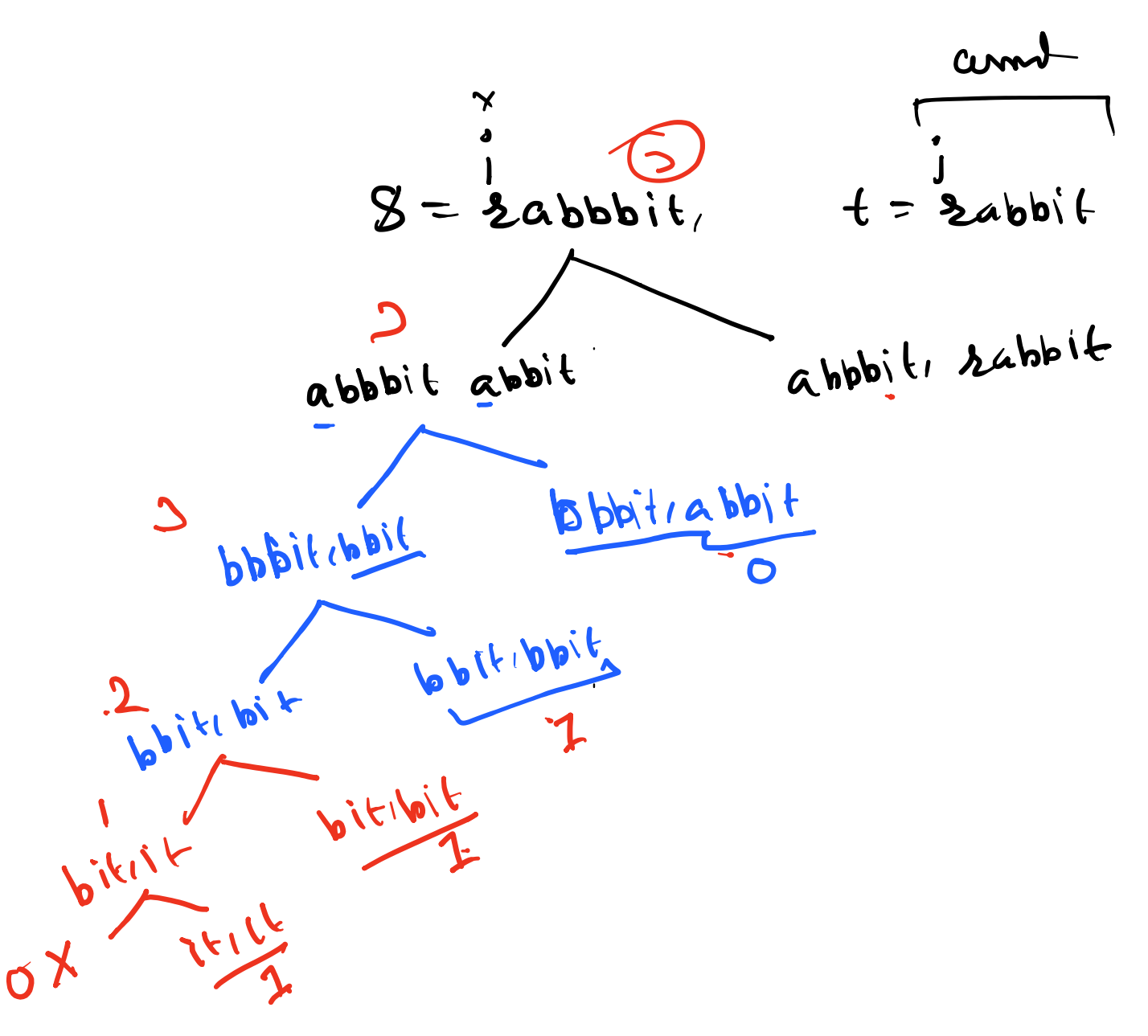
nums1 = [2,5,1,2,5]

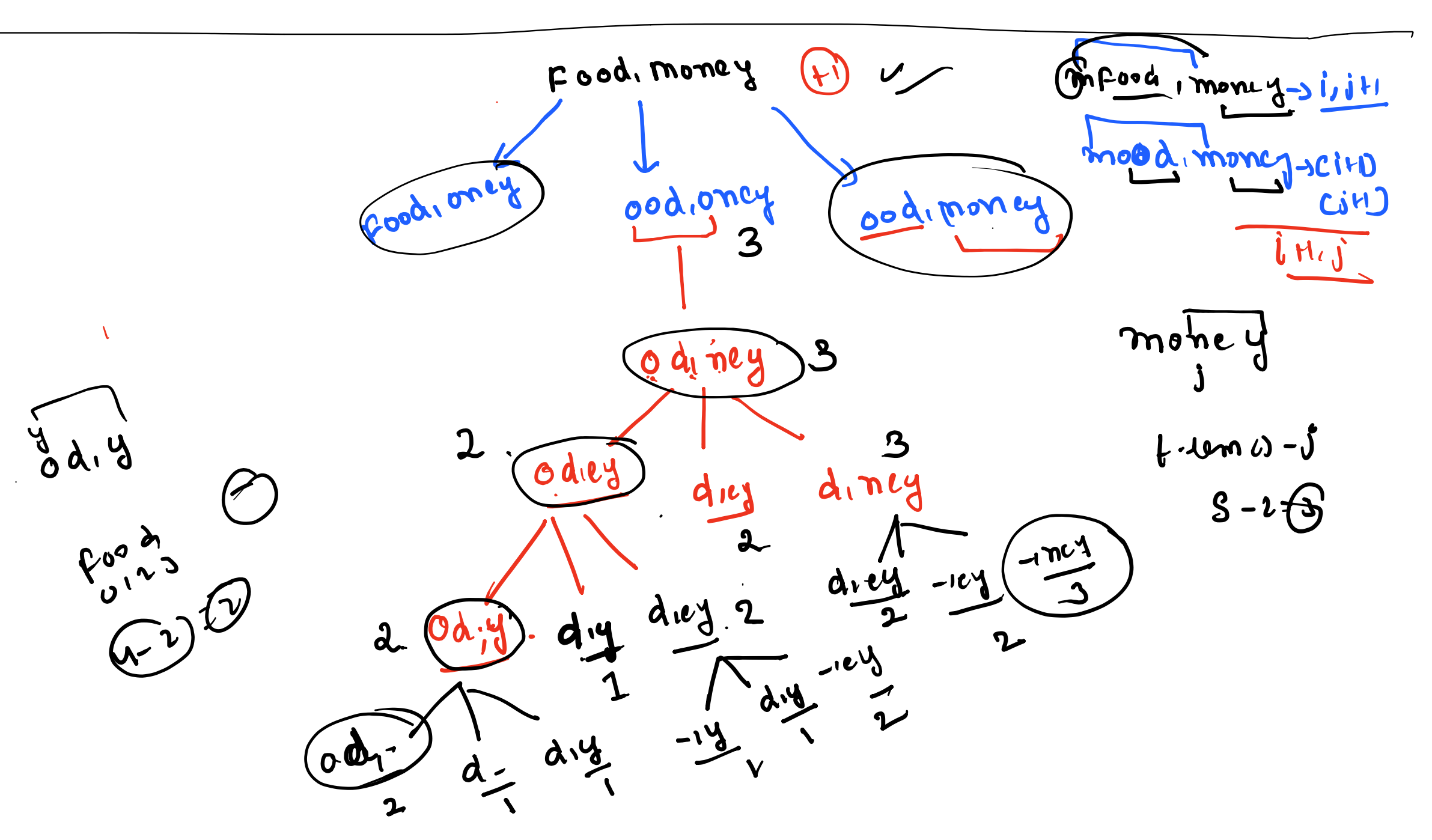
nums2 = [10,5,2,1,5,2]

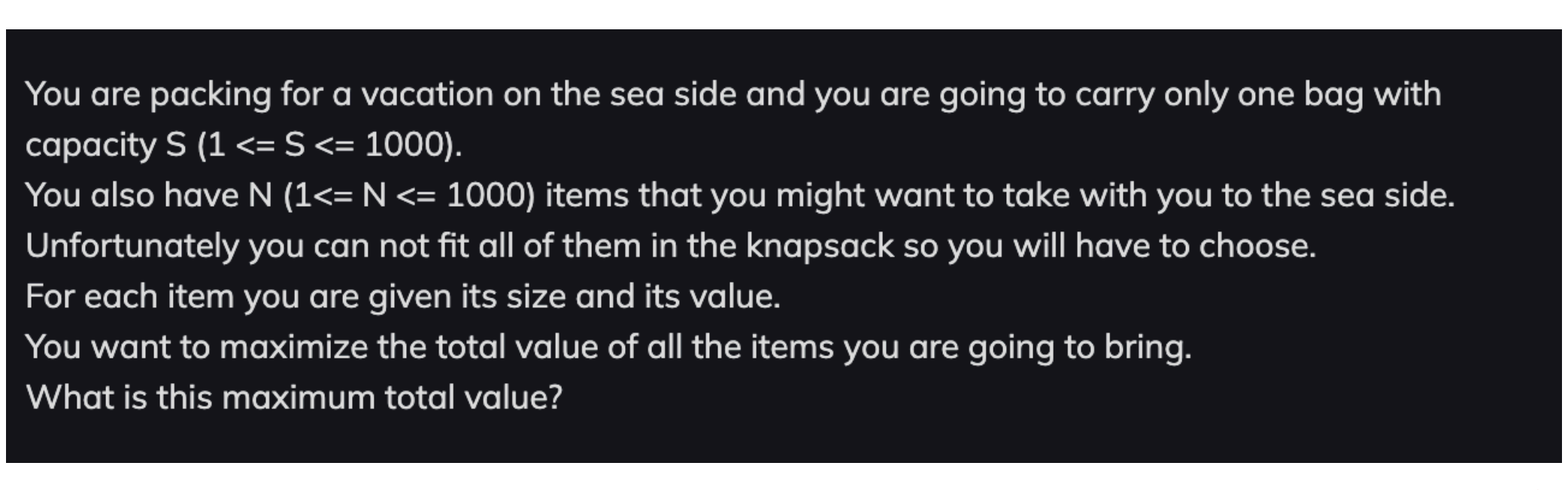

1 2 2 5 5
1 2 2 5 5 10


[1,2,5]
4
11111
4 [1,2,5]
5 [2,5]
3 [1,2,5]
4 [2,5]
2 [1,2,5]
3 [2,5]
1 [1,2,5]
2 [2,5]
0 [1,2,5]
1 [2,5] 0

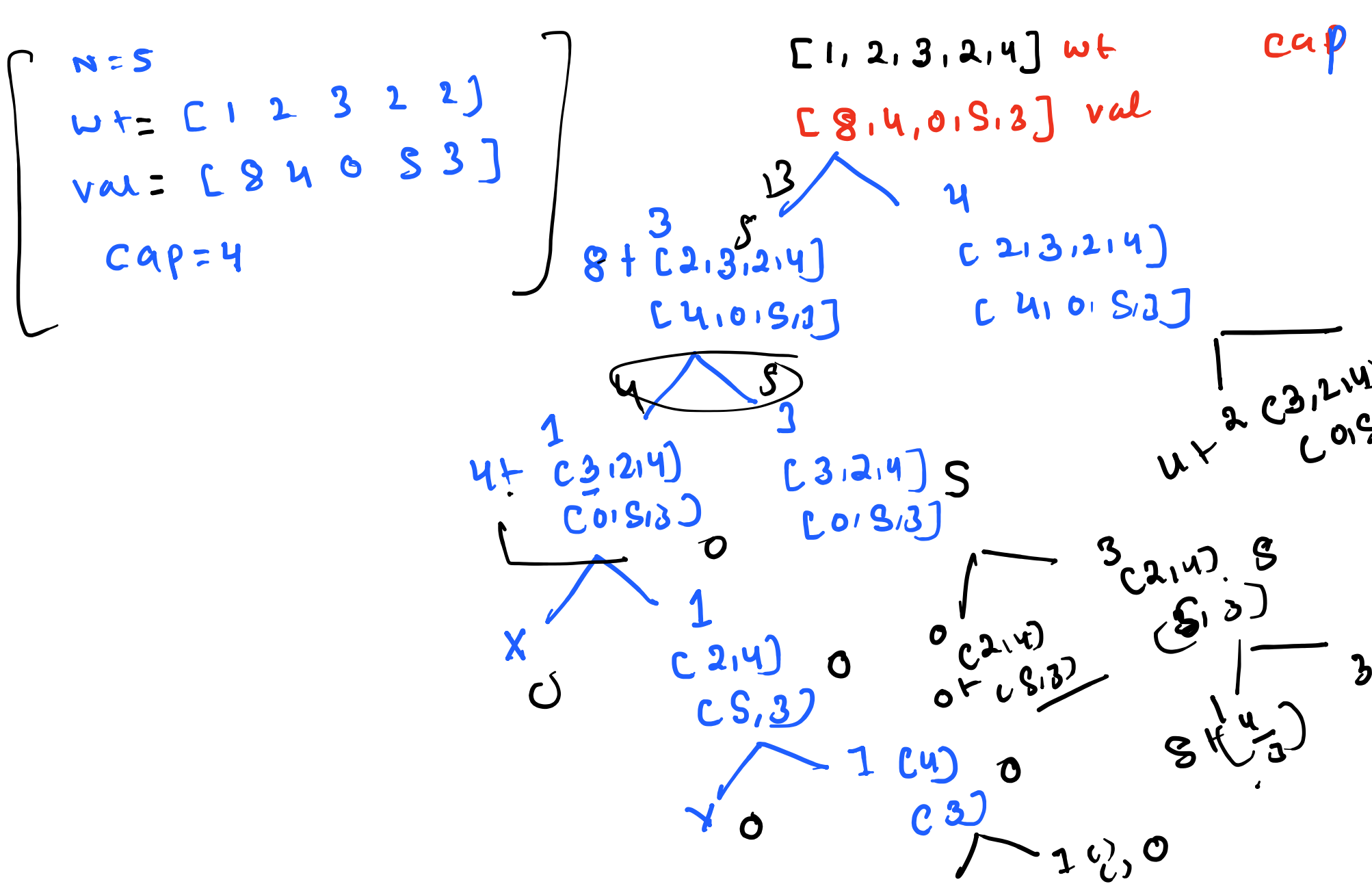

dp[i][j−cans]
dp[i−1][j]
i 0 1 2 3 4 5
5 [1,2,5]
0 [1,2]
5 [1,2]
1 [1,2,5]
2 [1,2,5]
3 [1,2,5]
4 [1,2,5]
[1,2,5] = ans

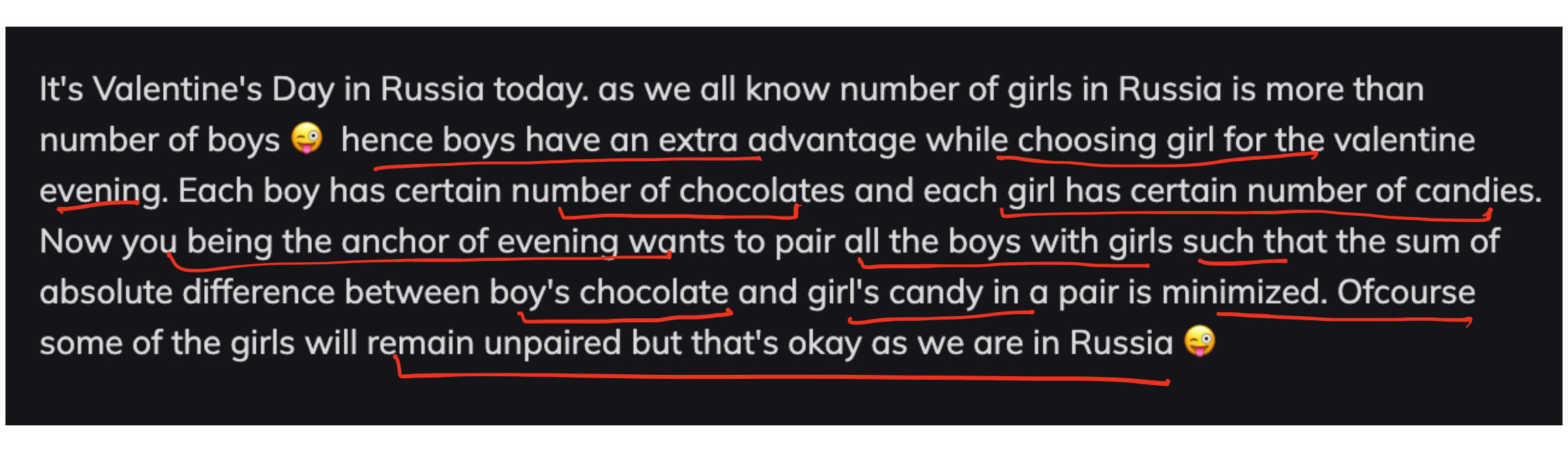

s = rabbbit,
t = rabbit
ans
abbbit abbit
abbbit, rabbit
bbbit, bbit
bbbit, abbit
bbit, bit
bbit, bbit
bit, bit
bit, bit
it, it


food, money
ood, oney
ood, money
food, oney
od, ney
odey
dey
dney
money
t.len() − j
S − 2 = 3

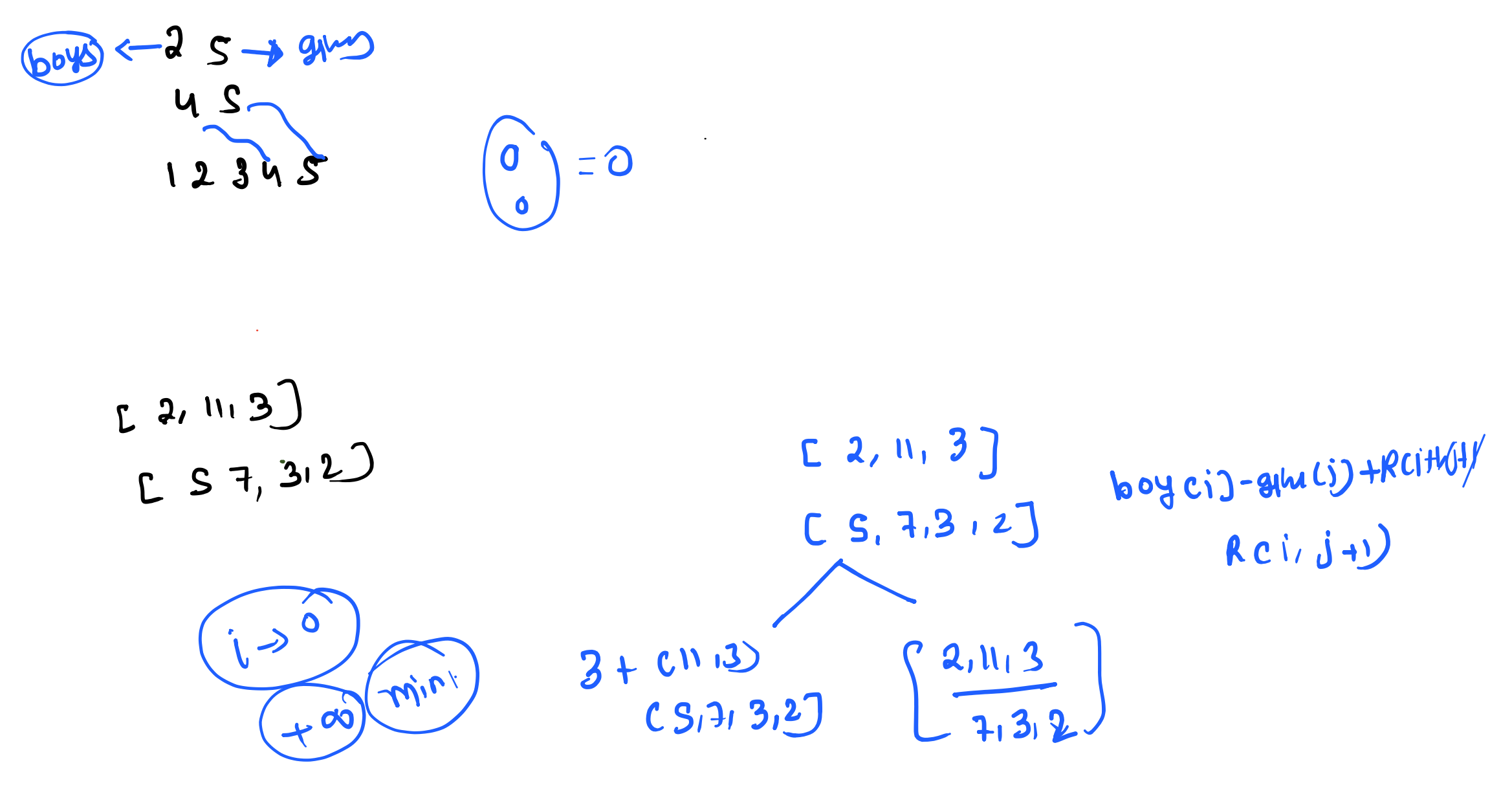


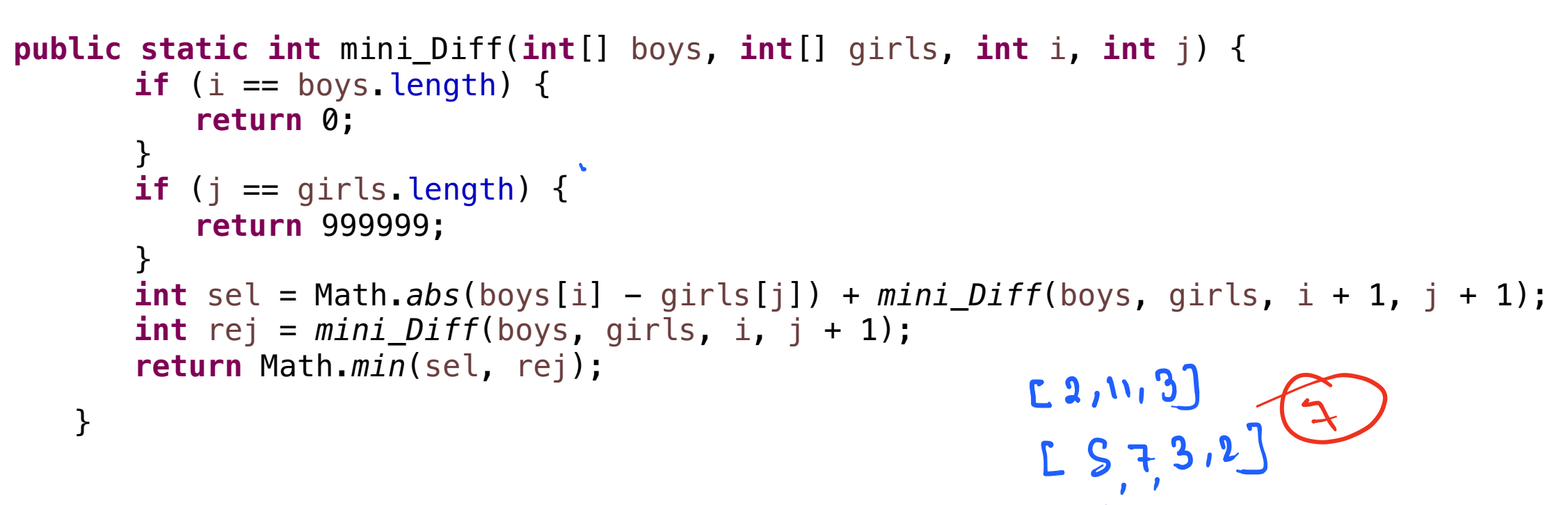
You are packing for a vacation on the sea side and you are going to carry only one bag with capacity S (1 <= S <= 1000).
You also have N (1<= N <= 1000) items that you might want to take with you to the sea side.
Unfortunately you can not fit all of them in the knapsack so you will have to choose.
For each item you are given its size and its value.
You want to maximize the total value of all the items you are going to bring.
What is this maximum total value?


N = 5
wt = [1 2 3 2 2]
val = [8 4 0 5 3]
cap = 4
[1, 2, 3, 2, 4] wt
[8, 4, 0, 5, 3] val
cap
8 + [2,3,2,4]
[4,0,5,3]
[3,2,4]
[0,5,3]
[2,4]
[5,3]

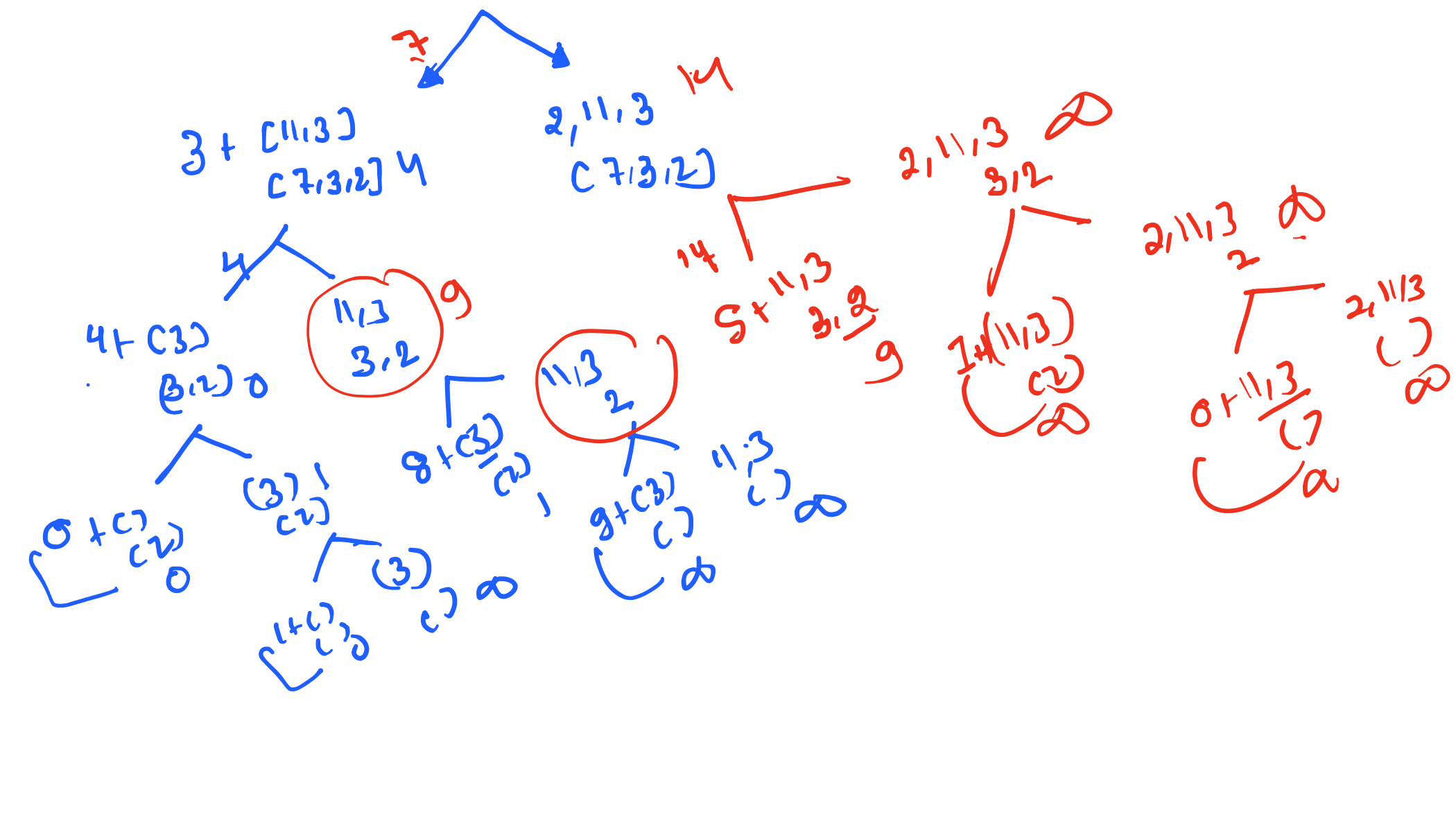


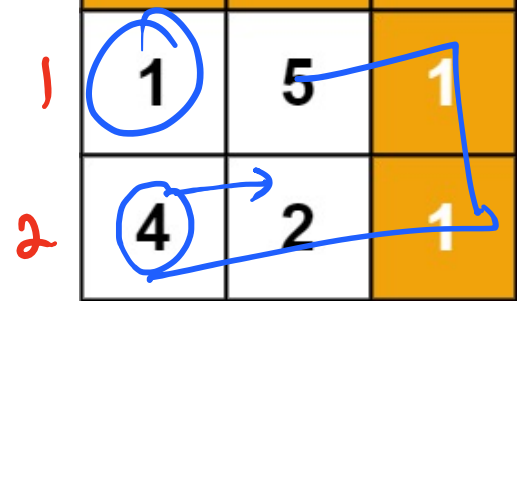
It's Valentine's Day in Russia today. as we all know number of girls in Russia is more than number of boys 😜 hence boys have an extra advantage while choosing girl for the valentine evening. Each boy has certain number of chocolates and each girl has certain number of candies. Now you being the anchor of evening wants to pair all the boys with girls such that the sum of absolute difference between boy's chocolate and girl's candy in a pair is minimized. Ofcourse some of the girls will remain unpaired but that's okay as we are in Russia 😜


boys ← 2 5 → girls
4 5
1 2 3 4 5
= 0
[2, 11, 3]
[5 7, 3, 2]
i → 0
+∞
min
boy[i] − girl[j] + R[i+1][j+1]
R[i, j+1]
3 + [11, 3]
[5, 7, 3, 2]
[2, 11, 3]
[7, 3, 2]

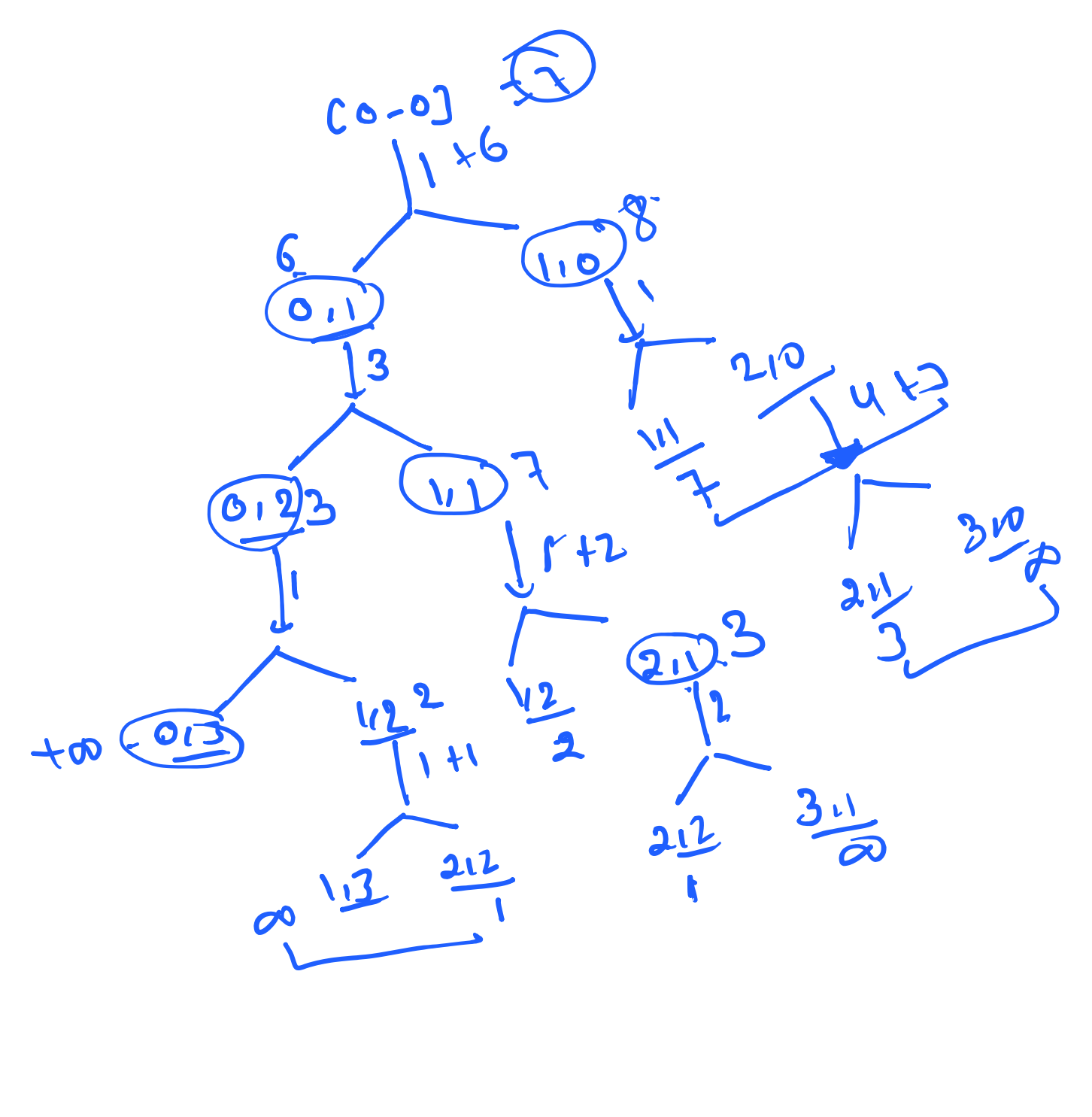


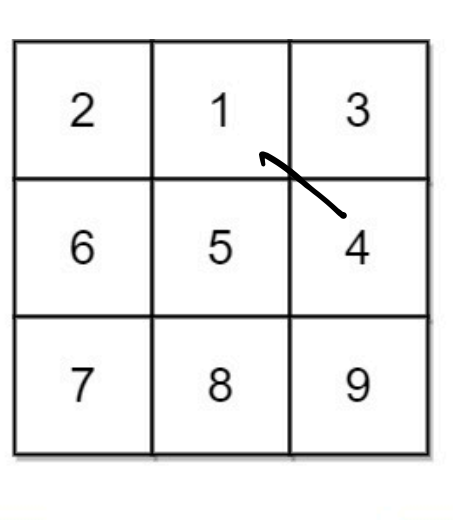
```
public static int mini_Diff(int[] boys, int[] girls, int i, int j) {
        if (i == boys.length) {
            return 0;
        }
        if (j == girls.length) {
            return 999999;
        }
        int sel = Math.abs(boys[i] - girls[j]) + mini_Diff(boys, girls, i + 1, j + 1);
        int rej = mini_Diff(boys, girls, i, j + 1);
        return Math.min(sel, rej);
    }
```


[2,11,3]
[5,7,3,2]
7
3 + [11,3]
[7,3,2]
2,11,3
[7,3,2]
14
∞

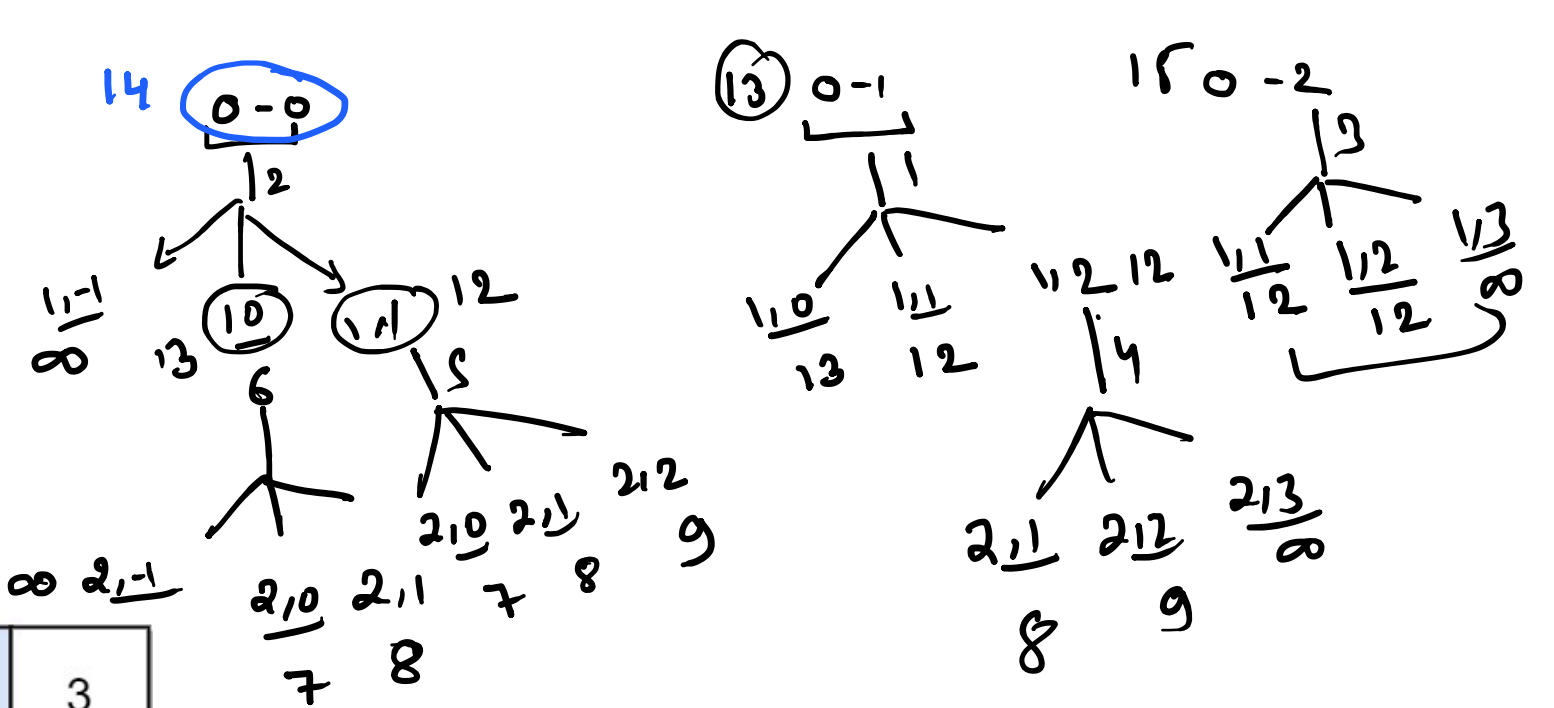


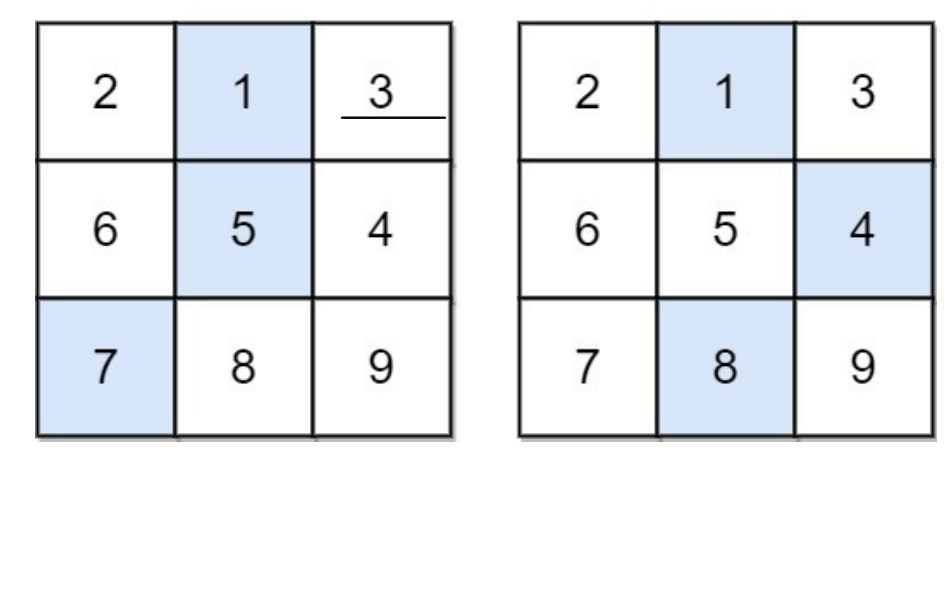
	0	1	2
0	1	3	1
1	1	5	1
2	4	2	1


[0−0] 7
1 + 6
0,1
1,0
0,2
1,1
2,0
2,1
3,0
1,2
2,2
0,3
1,3
3,1
+∞
∞



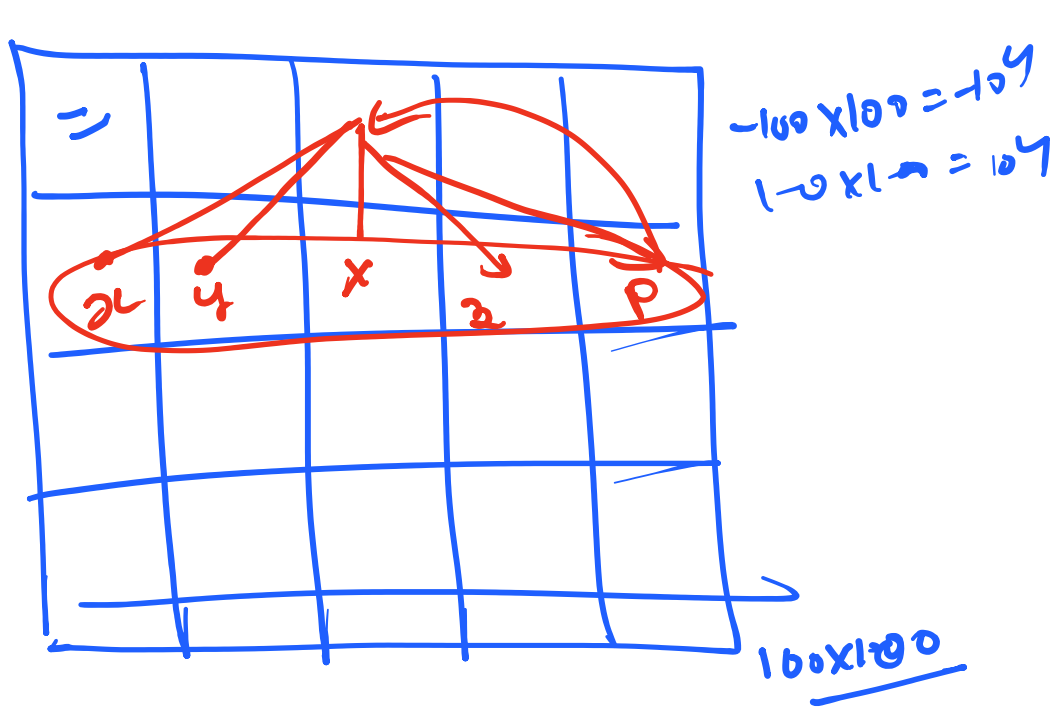
2	1	3
6	5	4
7	8	9


14 0−0
13 0−1
15 0−2
1,0
1,1
1,2
2,0
2,1
2,2
2,3
∞



2	1	3
6	5	4
7	8	9

2	1	3
6	5	4
7	8	9


−100 × 100 = 10^4
1 → ∞ × 1 ∞ = 10^4
100 × 100